

BioProtocolAgent:

    


                                                         

                                                         

                                                         

                                                         

                                                         

                                                         

                                                         

                            

[Verifier (optional)] → Chain-of-Verification ■■

↓

[Writer (optional)] → ■■■ ■■■■ ■■

GraphState

↓

[Task Planner] → ■■ Methods ■■■■■■ "■■■■(■■)" ■■

↓

[Step Planner] → ■ ■■■■■ "■■(■■)" ■■■■

↓

[Action Extractor] → ■ ■■■■ "■■" + ■■■■■/■■■■■ ■■

↓

[Condition Extractor] → ■■, ■■ ■ ■■ ■■

↓

[Verifier (optional)] → Chain-of-Verification ■■

↓

[Writer (optional)] → ■■■ ■■■■ ■■

■■■ ■■ ■■

1. Task Planner (`task\_planner.py`)

```
- **■■■**: ■■ Methods ■■■ ■■ ■■ ■■(task) ■■
- **LLM**: GPT-4-1106-preview (temperature=0.0, JSON mode)
- **■■■**: ■■ ■■, ■■■■ ■, Methods ■■■
- **■■■**: Task ■■■ (task_id, task_name, description, span_chunk)
- **■■■**:
  - ■■ ■■ ■■■ (relevant task■ ■■)
  - ■■/■■■ ■■■ ■■
```

2. Step Planner (`step\_planner.py`)

```
- **■■■**: Task■ ■■■ ■■(step)■ ■■■
- **LLM**: GPT-4o-mini (temperature=0.0)
- **■■■**: Task ■■ + Methods ■■■
- **■■■**: StepIR (step_id, step_text, step_rationale, evidence_spans)
- **■■■**: ■■■ JSON ■■ (■■■■ ■■, ■■ ■■)
```

3. Action Extractor (`action\_extractor.py`)

```
- **■■■**: Step■ ■■■ ■■(Add, Incubate, Mix, Centrifuge ■)■■ ■■
- **LLM**: GPT-4o-mini
- **■■■**: ActionIR ■■■
  ```python
 {
 "action": "Incubate",
 "description": "...",
 "materials": [{"name": "RNA", "role": "substrate", "volume": "100 µL"}],
 "conditions": [{"type": "temperature", "value": "37°C", "unit": "°C"}],
 "produces": ["purified RNA"],
 "evidence_span": "..."
 }
  ```
```

4. Condition Extractor (`condition\_extractor.py`)

```
- **■■■**: ■■, ■■, pH, ■■■(rpm) ■ ■■■ ■■ ■■
```

```

- **Tool**: (type, value, unit, source)
#### 5. Verifier Nodes (`verifier.py`, `verifier_react.py`)
- **Tool**: Chain-of-Verification (CoV) + ReAct
- Methods (supported, partially_supported, hallucinated, contradicted)
- **LLM**: GPT-4o-mini
- **Tool**: verification
```python
{
 "global_verdict": "supported",
 "reasoning_traces": [...],
 "revision_suggestion": "...",
 "order_suggestion": "no_change"
}
```

```

```

#### 6. Writer (`writer.py`)
- **Tool**: ActionIR →
- **LLM**:

```

```

### (`src/tools/`)

**rag_search.py**	Methods (FAISS)
**doc_search.py**	
**verifier_tools.py**	(, )
**calculate_tool.py**	,
**reorder_tool.py**	

```

```

---

##

### GraphState (LangGraph)

```



```

#### 1. Task Planner (task_planner.py)
- : Methods (task)
- LLM: GPT-4-1106-preview (temperature=0.0, JSON mode)
- : , Methods
- : Task (task_id, task_name, description, span_chunk)
- :
- (relevant task)
- /
#### 2. Step Planner (step_planner.py)
- : Task (step)
- LLM: GPT-4o-mini (temperature=0.0)
- : Task + Methods

```

- **StepIR** (step_id, step_text, step_rationale, evidence_spans)
- **JSON** (Add, Incubate, Mix, Centrifuge)

3. Action Extractor (action_extractor.py)

- **StepIR** (Add, Incubate, Mix, Centrifuge)
- **LLM**: GPT-4o-mini
- **ActionIR**

```
```python
```

```
{
 "action": "Incubate",
 "description": "...",
 "materials": [{"name": "RNA", "role": "substrate", "volume": "100 μL"}],
 "conditions": [{"type": "temperature", "value": "37°C", "unit": "°C"}],
 "produces": ["purified RNA"],
 "evidence_span": "..."
}
```

#### #### 4. Condition Extractor (condition\_extractor.py)

- **Condition** (pH, rpm)
- **Condition** (type, value, unit, source)

#### #### 5. Verifier Nodes (verifier.py, verifier\_react.py)

- **Chain-of-Verification (CoV) + ReAct**
- **Methods**
- **Verdict** (supported, partially\_supported, hallucinated, contradicted)
- **LLM**: GPT-4o-mini
- **verification**

```
```python
```

```
{
  "global_verdict": "supported",
  "reasoning_traces": [...],
  "revision_suggestion": "...",
  "order_suggestion": "no_change"
}
```

6. Writer (writer.py)

- **ActionIR** → **Condition**
- **LLM**: **Condition**

■■ ■■■ (`src/tools/`)

■ ■■■ ■■

GraphState (LangGraph ■■)

```
class GraphState(TypedDict, total=False):
    protocol_id: str # "Bio-protocol-2302"
    bio: Dict[str, Any] # {title, protocol_info}
    article: Dict[str, Any] # {title, pmcid, sections}
    methods_text: str # ■■ Methods ■■
    tasks: List[Dict] # Task Planner ■■
    steps_raw: List[StepIR] # Step Planner ■■
    steps: List[StepIR] # ■■■ steps
    actions: List[ActionIR] # Action Extractor ■■
    protocol_draft: str # ■■ ■■■■ ■■■
    tool_logs: List # ■■ ■■ ■■
    verification_logs: List # ■■ ■■
```

```
class GraphState(TypedDict, total=False):
    protocol_id: str # "Bio-protocol-2302"
    bio: Dict[str, Any] # {title, protocol_info}
    article: Dict[str, Any] # {title, pmcid, sections}
    methods_text: str # ■■ Methods ■■
    tasks: List[Dict] # Task Planner ■■
    steps_raw: List[StepIR] # Step Planner ■■
    steps: List[StepIR] # ■■■ steps
    actions: List[ActionIR] # Action Extractor ■■
    protocol_draft: str # ■■ ■■■■ ■■■
    tool_logs: List # ■■ ■■ ■■
    verification_logs: List # ■■ ■■
    ### ActionIR (■■ ■■ ■■)
```

ActionIR (■■ ■■ ■■)

```
class ActionIR(TypedDict, total=False):
    action_id: str # "Bio-protocol-2302::T3::S1::A1"
    action: str # "Add", "Incubate", "Mix"
    description: str # ■■■ ■■
    materials: List[MaterialIR] # ■■/■■
    conditions: List[ConditionIR] # ■■, ■■ ■
    produces: List[str] # ■■■
    evidence_span: str # Methods■■ ■■ ■■■
    source: str # "llm_extracted"
```

```
class ActionIR(TypedDict, total=False):
    action_id: str # "Bio-protocol-2302::T3::S1::A1"
```

```

action: str # "Add", "Incubate", "Mix"
description: str # ■■■■ ■■
materials: List[MaterialIR] # ■■/■■
conditions: List[ConditionIR] # ■■, ■■ ■
produces: List[str] # ■■■■
evidence_span: str # Methods■■ ■■ ■■■■
source: str # "llm_extracted"

```

```
---
```

```
## ■ ■■ ■■ ■■■■
```

```
### ■■ ■■ ■■ (ablation study)
```

```

| ■■■ | ■■ | ■■ |
|-----|-----|-----|
| **Step Precision** | ■■■ ■■ ■ ■■■ ■■ | 0.0 ~ 1.0 |
| **Step Recall** | ■■■■ ■■■ ■■ ■■ | 0.0 ~ 1.0 |
| **Step F1** | ■■■■ ■■■■ ■■■■ | 0.0 ~ 1.0 |
| **Order Score** | ■■ ■■ ■■■ | 0.0 ~ 1.0 |
| **Material IOU** | ■■ ■■ (Soft IOU) | 0.0 ~ 1.0 |
| **Condition IOU** | ■■ ■■ (Soft IOU) | 0.0 ~ 1.0 |
| **Grounding Hallucination Rate** | Methods■■ ■■ ■■ ■■ ■■ | 0.0 ~ 1.0 |
| **Evidence Coverage** | Methods ■■■ ■■■■ | 0.0 ~ 1.0 |

```

```
### ■■ ■■ ■■ (P1-P6 ■■ ■■■■)
```

```

| ■■■■ | n_pred | Step F1 | Order Score | Mat IOU | Cond IOU | Hallucination |
|-----|-----|-----|-----|-----|-----|-----|
| **P1** (■■■) | 15.6 | 0.494 | 0.510 | 0.247 | 0.573 | 0.25 |
| **P2** (■■■) | 8.6 | 0.317 | 0.428 | 0.318 | 0.319 | 0.00 |
| **P3** | 12.3 | 0.435 | 0.578 | 0.319 | 0.667 | 0.10 |
| **P4** | 12.9 | 0.454 | 0.534 | 0.192 | 0.680 | 0.25 |
| **P5** (■■ ■■) | 33.5 | **0.816** | 0.531 | **0.415** | 0.531 | 0.00 |
| **P6** (■■■) | 29.3 | **0.741** | **0.558** | **0.456** | 0.532 | 0.00 |

```

```
### ■■ ■■■■
```

```

- **P5, P6 (■■ ■■ ■ ■■■)**: Step F1 80% ■■ ■■
- **■■ ■■ ■■**: Condition IOU 53~68% (■■■■ ■■)
- **■■(Hallucination) ■■**: ■■ ■■■■■■ 0%
- **■■■■■■**: ■■■ ■■ ■ ■■ vs ■■■ ■■

```

```
---
```

```
## ■ ■■ ■■■■
```

```
### 1. Ablation Study (`eval_ablation_actions_v2.py`)
```

```

- **■■■**: Gold Action IR + ■■■ Action IR
- **■■■■■**:
  1. Step ■■ (Sentence-Transformer ■■, threshold=0.7)
  2. ■■/■■ ■■ (Soft IOU)
  3. Methods ■■ ■■ (■■ ■■ similarity)
  4. ■■ ■■ ■■

```

```
### 2. Hierarchical Protocol ■■ (`eval_hier_protocol_vs_generated.py`)
```

```
- **■■■**: ■■■ ■■■ ■■■■ gold protocol ■■
```

```

- **eval**:
- BLEU score
- ROUGE score
- Step-level F1 (Step F1)
- TF-IDF cosine similarity

### 3. Grounding (eval_methods_grounding_protocols.py)
- **eval**: eval Methods (1200, 200)
- **eval**: Methods (1200, 200)

---

## LLM

| Task/Step Planning | GPT-4 | gpt-4-1106-preview | JSON mode |
|-----|-----|-----|-----|
| Action/Condition | GPT-4o-mini | gpt-4o-mini | JSON mode |
| (CoV+ReAct) | GPT-4o-mini | gpt-4o-mini | JSON mode |
| (Writer) | GPT-4o-mini | gpt-4o-mini | JSON mode |

###
- **JSON Mode**: JSON schema
- **Temperature=0.0**: (0.0)
- **Response Format**: JSON schema

---

##

```

- P5, P6 (Step F1 80%)
- Condition IOU 53~68% ()
- (Hallucination) : 0%
- : vs

###

- **IR**: Gold Action IR + **IR** Action IR
- **IR**:
- 1. Step **IR** (Sentence-Transformer **IR**, threshold=0.7)
- 2. **IR**/**IR** **IR** (Soft IOU)
- 3. Methods **IR** **IR** (**IR** **IR** similarity)
- 4. **IR** **IR** **IR**

2. Hierarchical Protocol **IR** (`eval\_hier\_protocol\_vs\_generated.py`)

- **IR**: **IR** **IR** **IR** gold protocol **IR**
- **IR**:
- BLEU score
- ROUGE score
- Step-level F1 (**IR** **IR**)
- TF-IDF cosine similarity

3. Grounding **IR** (`eval\_methods\_grounding\_protocols.py`)

- **IR**: **IR** **IR** Methods**IR** **IR** **IR** **IR**
- **IR**: Methods**IR** **IR** **IR** (1200**IR**, 200**IR** **IR**) **IR** **IR**

IR LLM **IR** **IR**

IR **IR**

- **JSON Mode**: **IR** **IR** **IR**
- **Temperature=0.0**: **IR** **IR** (**IR** **IR**)
- **Response Format**: JSON schema **IR**

IR **IR** **IR** **IR**

```
BioProtocolAgent/
├── main.py                # IR IR
├── src/
│   ├── types.py           # GraphState, ActionIR, StepIR IR
│   ├── graph_builder.py   # LangGraph IR
│   ├── data_loader.py     # JSONL IR IR
│   ├── nodes/
│   │   ├── task_planner.py # Task IR
│   │   ├── step_planner.py # Step IR
│   │   ├── action_extractor.py # Action IR
│   │   ├── condition_extractor.py # IR IR
│   │   └── verifier.py     # CoV IR
```



```

■ ■ ■■■ verifier_react.py # ReAct ■■
■ ■ ■■■ writer.py # ■■■ ■■
■ ■ ■■■ order_structurer.py # ■■ ■■■
■ ■■■ tools/ # ■■ ■■■
■ ■ ■■■ rag_search.py # ■■ ■■
■ ■ ■■■ verifier_tools.py # ■■ ■■
■ ■ ■■■ ...
■ ■■■ eval/ # ■■ ■■
■ ■■■ logs/ # ■■ ■■
■■■ scripts/ # ■■ ■■■■
■ ■■■ eval_ablation_actions_v2.py # Ablation ■■
■ ■■■ eval_hier_protocol_vs_generated.py # ■■■■ ■■
■ ■■■ eval_methods_grounding_protocols.py # Grounding ■■
■ ■■■ gen_protocols_from_actions_llm.py # IR → ■■■■
■ ■■■ ...
■■■ data/
■ ■■■ gold_pairs_testset_v2.jsonl # ■■■■
■ ■■■ gold_actions_ir_10.jsonl # Gold ■■
■ ■■■ ablation/ # ■■■■ ■■
■ ■■■ ...
■■■ reports/ # ■■ ■■
■ ■■■ protocol_eval/ # ■■■■ ■■
■ ■■■ grounding_eval/ # Grounding ■■
■ ■■■ llm_protocols/ # ■■■■ ■■■■
■ ■■■ hallucination_report_*.csv
■■■ notebooks/ # ■■■■ & ■■

```

BioProtocolAgent/

```

■■■ main.py # ■■■ ■■■
■■■ src/
■ ■■■ types.py # GraphState, ActionIR, StepIR ■■
■ ■■■ graph_builder.py # LangGraph ■■
■ ■■■ data_loader.py # JSONL ■■■ ■■
■ ■■■ nodes/ # ■■■■ ■■■■
■ ■ ■■■ task_planner.py # Task ■■
■ ■ ■■■ step_planner.py # Step ■■■
■ ■ ■■■ action_extractor.py # Action ■■
■ ■ ■■■ condition_extractor.py # ■■ ■■
■ ■ ■■■ verifier.py # CoV ■■
■ ■ ■■■ verifier_react.py # ReAct ■■
■ ■ ■■■ writer.py # ■■■ ■■
■ ■ ■■■ order_structurer.py # ■■ ■■■■
■ ■■■ tools/ # ■■ ■■■■
■ ■ ■■■ rag_search.py # ■■ ■■
■ ■ ■■■ verifier_tools.py # ■■ ■■
■ ■ ■■■ ...
■ ■■■ eval/ # ■■ ■■
■ ■■■ logs/ # ■■ ■■

```

```
scripts/ #
eval_ablation_actions_v2.py # Ablation
eval_hier_protocol_vs_generated.py #
eval_methods_grounding_protocols.py # Grounding
gen_protocols_from_actions_llm.py # IR →
...
data/
gold_pairs_testset_v2.jsonl #
gold_actions_ir_10.jsonl # Gold
ablation/ #
...
reports/ #
protocol_eval/ #
grounding_eval/ # Grounding
llm_protocols/ #
hallucination_report_*.csv
notebooks/ # &

---

##
### 1. (Hierarchical Decomposition)
```



1. (Hierarchical Decomposition)

```
Protocol
  Task
    Step
      Action ( )

Protocol
Task
Step
Action ( )
  LLM

### 2. Chain-of-Verification (CoV)
-
-
-

### 3.
```

2. ****LLM** ******

```

- ##### (JSON schema)
- ## (hallucination)
- (temperature) /

3. ** & RAG**
- Sentence-Transformers
- FAISS
- Soft matching

4. ** **
- F1, Precision, Recall ( )
- IOU ( / )
- Order score ( )
- Grounding hallucination rate ( )

### /
1. ** **: Methods →
2. ** **: (evidence_span)
3. ** **: CoV
4. **Ablation Study**: P1~P6

###
- **Python 3.12**, (Pydantic)
- ** **: JSON
- ** **: (LangGraph)
- ** **: Temperature=0.0, seed

---

##  4

### (P5, P6)
- **Step F1**: 81.6% (P5), 74.1% (P6)
- **Material IOU**: 41.5% (P5), 45.6% (P6)
- **Condition IOU**: 53.1% (P5), 53.2% (P6)
- ** **: 0% (P5, P6)

###
- ** **: 10 Bio-protocol
- **Baseline**: P1 ( )
- ** **: P5~P6 ( , CoV )

---

##  4
- ** **: ( `cabb52b: "Feat:  "`)
- ** **:
- ** **: ( , , , )

---

##  4 ( )

```





1. LangGraph ■■ ■■■■■■ ■■■■■■■■

- Stateful workflow (GraphState)
- ■■■■ ■■■■ (if-else, dynamic routing)
- ■■ ■■■■ (LangGraph Checkpoint)

2. LLM ■■■■ ■■■■■■

- ■■■■ ■■ (JSON schema)
- ■■ ■■ ■■■■ (hallucination ■■)
- ■■(temperature) ■■■■ ■■■■/■■■■ ■■

3. ■■ ■■■■ & RAG

- Sentence-Transformers■ ■■ ■■ ■■
- FAISS■ ■■■■ ■■
- Soft matching■■ ■■■■ ■■ ■■

4. ■■ ■■■■ ■■

- F1, Precision, Recall (■■ ■■)
- IOU (■■/■■ ■■)
- Order score (■■ ■■■■)
- Grounding hallucination rate (■■■)



1. ■■ ■■ ■■■■: Methods → ■■ ■■■■ ■■■■
2. ■■ ■■■■: ■■ ■■ (evidence_span)
3. ■■ ■■ ■■: CoV■ ■■ ■■ ■■
4. Ablation Study: P1~P6■ ■■■■ ■■ ■■



- Python 3.12, ■■ ■■■■ (Pydantic)
- ■■ ■■: JSON ■■ ■■ ■ ■■ ■■ ■■
- ■■■■: ■■■■ ■■ ■■ ■■ (LangGraph ■■)
- ■■■■: Temperature=0.0, seed ■■



■■ ■■ ■■ (P5, P6)

- Step F1: 81.6% (P5), 74.1% (P6)
- Material IOU: 41.5% (P5), 45.6% (P6)

- **Condition IOU:** 53.1% (P5), 53.2% (P6)
- **■■■:** 0% (P5, P6)

- ■■■■: 10■ Bio-protocol
- **Baseline:** P1 (■■ ■■■■)
- ■■■■: P5~P6 (■■ ■■, CoV ■■)

- ```
- 00000000: 00000000 (cabb52b: "Feat: 000 000 000 000")
- 00000001: 00000000 00000000 00000000 00000000
- 00000002: 0000 0000 (000, 000, 000, 000)
```

```
■■■■ ■■■■■■
langgraph==1.0.4
langchain==1.1.0
langchain-core==1.1.0
```

```
🐳 & 🐳 🐳
sentence-transformers==5.1.2
faiss-cpu==1.12.0
torch==2.9.0
transformers==4.57.1
```

```
■■■ ■■
sacrebleu
rouge-score
```



langgraph==1.0.4  
langchain==1.1.0  
langchain-core==1.1.0

## LLM API

openai==2.6.0  
pydantic==2.12.3



sentence-transformers==5.1.2  
faiss-cpu==1.12.0  
torch==2.9.0  
transformers==4.57.1



scikit-learn  
scipy  
pandas  
numpy



sacrebleu  
rouge-score



python-dotenv==0.9.9

---

## ■■ ■■ ■■ & ■■

### ■■





```

source .venv/bin/activate

echo "OPENAI_API_KEY=sk-..." > .env

python main.py
```



source .venv/bin/activate



echo "OPENAI\_API\_KEY=sk-..." > .env



python main.py

```
##
```



```
python scripts/eval_ablation_actions_v2.py
python scripts/eval_hier_protocol_vs_generated.py
python scripts/eval_methods_grounding_protocols.py
```

python scripts/eval\_ablation\_actions\_v2.py

python scripts/eval\_hier\_protocol\_vs\_generated.py

python scripts/eval\_methods\_grounding\_protocols.py

---

```
■ ■■ Q&A ■■
```

```
**Q: #####? **
```

```
A: LangGraph■ StateGraph■ 5■ ■■■ ■■/■■■ ■■■■■. GraphState■ ■■■ ■■■■, ■ ■■■ ■■■ LLM ■■■ ■■ ■■■
```

```
**Q: ■■(hallucination)■ ■■■ ■■■■? **
```

```
A: (1) ■■ ■■ ■■■■■ ■■ Task■ ■■, (2) Chain-of-Verification■■ ■■ ■■, (3) Methods ■■ ■■■■ Grounding ■■■■■
```

```
**Q: ■■■ ■■■ ■■■■? **
```

```
A: Step-level F1 (■■ ■■■), Material/Condition IOU (■■ ■■), Grounding hallucination rate (■■■■)■ ■■■■■. P
```

```
**Q: ■■■■ ■■■■ ■■■■? **
```

```
A: FAISS■ ■■■ ■■ ■■■ 0(n) → 0(log n)■■ ■■■■■, LangGraph■ ■■ ■■■■■ ■■ ■■■■ ■■■■■■.
```

---

```
■■■ ■■■: 2026-07-03
```



\*■■■■: BioProtocolAgent (■■ ■■ ■■)\*

---

## ■■ ■■ Q&A; ■■

Q: ■■■■■■ ■■■■ ■■■ ■■■■■■?

A: LangGraph■■ StateGraph■■ ■■■■ 5■■ ■■■■ ■■/■■■ ■■■■■■. GraphState■■ ■■■■ ■■■■, ■■■■ ■■■■ LLM ■■■■ ■■ ■■■■ ■■■■.

Q: ■■(hallucination)■■ ■■■■ ■■■■?

A: (1) ■■ ■■ ■■■■■■ ■■ Task■■ ■■, (2) Chain-of-Verification■■ ■■ ■■, (3) Methods ■■ ■■■■ Grounding ■■■■■■.

Q: ■■■■ ■■■■ ■■■■■■?

A: Step-level F1 (■■■ ■■■■), Material/Condition IOU (■■■ ■■■), Grounding hallucination rate (■■■)■■ ■■■■■■. P5/P6■■■ F1 80% ■■ ■■■■■■.

Q: ■■■■ ■■■■ ■■■■■■?

A: FAISS■■ ■■■■ ■■ ■■■■  $O(n) \rightarrow O(\log n)$ ■■ ■■■■■■, LangGraph■■ ■■ ■■■■■■ ■■ ■■■■ ■■■■■■.

---

■■■■ ■■■■: 2026-07-03

■■■■: BioProtocolAgent (■■ ■■ ■■)